Performance & Load Test Report

StackStorm e-commerce platform — Backend API | Generated 2026-11-03

Dedicated load/performance testing was not previously wired into the project's CI pipeline. This report was produced by running `autocannon` directly against the live backend API (Node/Express, PostgreSQL 16, Redis 7) in this sandbox, plus a custom concurrency script targeting the checkout/stock-reservation endpoint. **Scope note:** this is a local single-instance sandbox, not the production or staging environment, and results should be treated as a baseline rather than a capacity guarantee — but every number below is a real measured result, not an estimate.

1. HTTP load test results (autocannon, 15s / 20 connections each)

Endpoint	Path	p50	p97.5	p99	Avg throughput	Volume
Health check	GET /api/health	4ms	12ms	16ms	3,768 req/s	57k reqs in 15.0s
Product listing	GET /api/v1/products?limit=20	78ms	112ms	127ms	246 req/s	4k reqs in 15.0s
Product detail	GET /api/v1/products/:slug	4ms	9ms	12ms	3,893 req/s	58k reqs in 15.0s
Categories (cached)	GET /api/v1/categories	5ms	9ms	10ms	3,645 req/s	~55k reqs in 15.0s

1.1 Finding: product listing is the one real bottleneck

GET /api/v1/products runs at p50 78ms / p99 127ms and ~246 req/s, roughly 15–20× slower than every other endpoint tested (health, product detail and categories all sit at single-digit-millisecond p50 and 3,600–3,900 req/s). Product detail and categories are fast because they resolve a single row or hit the Redis cache (`cache.service.js`); listing is the only one of the four doing filtered, paginated aggregation across the product/variant/category tables on every request with no cache in front of it.
Recommendation: add a short-TTL cache for common listing queries (default sort, first page, no filters) the way `category.service.js` already does for categories, and check for a missing composite index on the filter/sort columns used by `product.service.js`'s listing query before the next sprint.

2. Concurrency / correctness under load: stock reservation

ADR-004 specifies pessimistic row-locking (`SELECT ... FOR UPDATE` with ascending-id lock ordering) for the checkout stock-reservation step, to guarantee no two concurrent checkouts can oversell the same variant. This was verified under real concurrent load rather than by reading the code alone.

2.1 Test design

A product variant was seeded with exactly **12** available units (`stock_quantity=12, reserved_quantity=0`). A custom Node script fired **30 concurrent** `POST /api/v1/checkout/initiate` requests (1 unit each) against the live backend using a real authenticated customer session, with no artificial delay between them, and recorded every response status and the final database state.

2.2 Result

Metric	Result
Concurrent requests fired	30
Succeeded (201 Created)	12
Correctly rejected (409 Conflict)	18
Elapsed time (all 30 requests)	247 ms
Oversold units	0

Final DB state (product_variants id=10)	reserved_quantity = 12, matching stock_quantity = 12

Sample rejection body returned to the 18 unsuccessful requests:
`{"error":"CONFLICT","message":"INSUFFICIENT_STOCK for variant DFC-4-M. Available: 0, Requested: 1"}`

Conclusion: exactly as many requests succeeded as there were units of stock, with zero oversell and no partial/inconsistent state, confirming the row-lock reservation design in `reservation.service.js` holds correctly under real concurrent load, not just in the unit tests that mock the database layer.

3. Methodology & environment

Backend served by Node/Express against local PostgreSQL 16 and Redis 7 (no Docker daemon available in this sandbox; both run as native services, same schema/migrations as production). Load generated with `autocannon` (15s duration, 20 connections, default pipelining) against each endpoint in turn, not simultaneously, to isolate per-endpoint latency. The concurrency test used Node's built-in `http` module to fire all 30 requests via `Promise.all` with a single shared bearer token from a freshly registered test customer. All figures in this report are taken directly from those runs on 2026-11-03.

4. Limitations

• Single sandbox instance, not a multi-instance/production topology — connection-pool and load-balancer behaviour under load is not exercised here.
• No sustained soak test (minutes/hours) was run; all tests were 15 seconds or a single 30-request burst.
• Stripe calls use the mock provider in this environment, so payment-gateway latency under load is not represented in these numbers.